```
def calculator
  if complete?
    CompleteTaskCalculator.new(self)
  else
    IncompleteTaskCalculator.new(self)
  end
end

def calculate
  calculator.calculate_time
end
```

In the most recent example there is still an if statement about the logic between complete and incomplete tasks, but only one. Any further difference between complete and incomplete tasks is handled in the difference between the two calculator classes. If you're testing for completion in many places, separating functionality into classes can be more clear.

Since these classes typically don't depend on the database, it's easy to write fast tests for them, and you'll often find these classes attracting behavior. Once they exist, it's easier to see what behavior belongs there.

A Note on Assertions per Test

You'll often find that a common setup results in multiple assertions in a test. This is particularly true of integration tests. For example, when you wrote your initial test to add tasks back in Chapter 3, *Test-Driven Rails*, on page 41, you made several assertions about the existence of the project and its relationship with the tasks you'd just created.

There are two contrasting styles for writing tests with multiple assertions. In one style, the setup and all the assertions are part of the same test. If you were trying to test changes when you mark a task complete, then having all the assertions in the same test might look like this:

```
it "marks a task complete" do
  task = tasks(:incomplete)
  task.mark_complete
  expect(task).to be_complete
  expect(task).to be_blocked
  expect(task.end_date).to eq(Date.today.to_s(:db))
  expect(task.most_recent_log.end_state).to eq("completed")
end
```

In contrast, you could put each assertion in a separate test and put the common setup in a setup block. Using the same set of assertions in separate tests looks like the following: